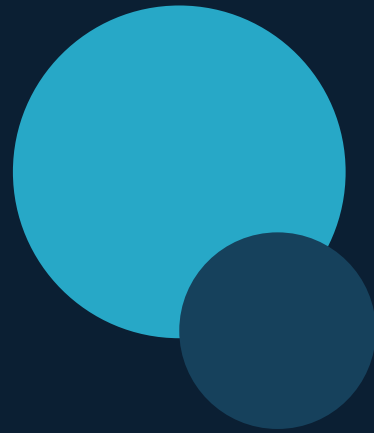


AI-based Village Pond Planning System



FINAL TECHNICAL REPORT

Assignment 1 | Phase 3 submission

Student: **Sneha Nagmoti**

Date: 28 August 2026

github.com/snehanagmoti/Village-Pond-planning

Contents

1. Abstract	3
2. Requirement traceability	3
3. System architecture	3
4. Phase 2 contour-file analysis	4
4.1 Input and validation	4
4.2 Terrain reconstruction	4
4.3 Hydrology and candidate selection	4
4.4 Provided-map result	4
5. Location-based elevation and hydrology	5
6. Rainfall analysis	5
7. Satellite surface screening and land limits	5
8. Runoff and pond geometry	6
9. Frontend and visualization	6
10. API, errors, and security	6
11. Reliability and operations	6
12. Verification evidence	7
13. Deployment and submission	7
14. Limitations and required field work	8
15. AI-tool usage and academic integrity	8
16. Conclusion	8
References	8

1. Abstract

This project implements a web-based decision-support system for preliminary village pond planning. It provides two connected workflows. The Phase 2 workflow accepts an arbitrary KML/KMZ contour map, reconstructs a terrain grid from the uploaded geometry, identifies a hydrologically strong candidate point, and delineates its contributing catchment. The complete application also supports a location-based workflow that combines satellite imagery, elevation, historical rainfall, surface screening, D8 hydrology, runoff estimation, and approximate pond geometry in an interactive map.

The implementation is deliberately transparent about uncertainty. Results carry source provenance, quality status, limitations, and a screening-only flag. The software does not claim government ownership, legal land availability, surveyed terrain accuracy, or construction safety. Those decisions require cadastral records, field survey, soil and groundwater investigation, qualified engineering design, and statutory approval.

2. Requirement traceability

Assignment requirement	Implemented evidence
Satellite imagery	Leaflet satellite base map and bounded imagery mosaic service
Contour visualization	Live DEM contour overlays and uploaded contour study boundary
Suitable/available land	Conservative bare-surface candidate; ownership explicitly unverified
Catchment area	Priority-flood conditioning, D8 routing, reverse catchment traversal, area in ha
Historical rainfall	Complete-year Open-Meteo ERA5-Land analysis with a validated NASA POWER fallback
Runoff volume	$V = C \times A \times P$ with visible coefficient and documentary basis
Pond depth/capacity	Side-sloped rectangular-frustum screening geometry with freeboard
Selected point and maps	Leaflet point, catchment, study area, candidate land and contour layers
KML/KMZ backend route	POST /api/analyze-contour plus assignment-compatible aliases
Structured result	Strict Pydantic JSON models, stable errors and OpenAPI documentation
Generalized implementation	All locations, geometry, elevations and outputs derived from input
Accessible frontend	Responsive keyboard-friendly React UI with explicit status/warnings
Installation/API docs	README, docs/API.md, docs/DEPLOYMENT.md, Swagger and ReDoc
Testing/code quality	Automated unit/API/UI tests, lint, coverage, audits, migrations and builds

3. System architecture

The browser client is a Vite/React single-page application using Leaflet for map display. It sends either a multipart contour upload or a location/radius JSON request to a FastAPI backend. Axios cancellation and a monotonically increasing request sequence prevent stale responses from replacing newer analysis.

FastAPI validates all public contracts with Pydantic and delegates geospatial work to independent services. Uploaded contours pass through a safe parser, terrain-grid reconstruction, and the shared hydrology pipeline. Location analysis uses independent elevation, rainfall, imagery, geocoding, terrain, runoff, and pond modules. CPU-heavy OpenCV and terrain work runs outside the asynchronous event loop.

PostgreSQL history is optional and disabled by default because it stores precise locations. When enabled, history requires an API key, bounded queries, Alembic migrations, and non-default credentials.

```

React / Leaflet
|-- KML/KMZ multipart --> FastAPI --> safe parser --> interpolated DEM
|                                     ^--> D8 catchment JSON
|-- location + radius --> FastAPI --> elevation + rainfall + imagery
|                                     ^--> runoff + pond screen

```

```

Optional protected history -----> PostgreSQL

```

4. Phase 2 contour-file analysis

4.1 Input and validation

The canonical route is `POST /api/analyze-contour`; the multipart field name is `contour_file`. Hidden compatibility aliases `/api/analyzeContour` and `/api/findCatchment` accept the same request. The route reads at most the configured 15 MiB plus one byte, closes the temporary upload, and runs CPU work in a worker thread.

The parser supports plain KML and KMZ. It obtains a line elevation, in order, from a numeric Placemark name, an elevation-like `Data` or `SimpleData` field, or a constant altitude coordinate. It accepts namespace variants and `MultiGeometry` because it traverses Placemark geometry recursively.

Validation rejects malformed XML, DTD/entity declarations, invalid coordinates, encrypted archives, unsafe compression ratios, too many archive entries, large expanded KMZ content, excessive contour/point counts, fewer than three lines or levels, and less than 0.5 m total relief. An uploaded polygon supplies a study boundary; otherwise a buffered convex hull is derived from the input contours.

4.2 Terrain reconstruction

KML stores vector isolines rather than a complete elevation raster. The service projects the study extent to a latitude-corrected metric grid. Grid dimensions are selected from input extent and contour interval, then bounded by configuration. Each contour is rasterized into fixed observed cells. Unknown interior cells are initialized from neighboring observations and solved by iterative harmonic interpolation while observed values remain fixed. The result records observed-cell ratio, iteration count, convergence, cell size, and method.

The study boundary masks both interpolation and later hydrology so cells outside the uploaded domain cannot become part of the catchment. Results are always labelled degraded, even on successful convergence, because an interpolated surface is not equivalent to a surveyed DEM.

4.3 Hydrology and candidate selection

The shared deterministic pipeline performs:

- 1 finite grid and analysis-mask validation;
- 2 priority-flood depression filling;
- 3 a very small deterministic gradient for equal-elevation flats;
- 4 steepest lower-neighbor D8 flow direction;
- 5 topological upstream flow accumulation;
- 6 candidate selection by maximum contributing area, with lower elevation as the tie-breaker;
- 7 reverse graph traversal to collect all cells draining to that candidate;
- 8 latitude-corrected catchment-area calculation; and
- 9 simplified WGS84 boundary extraction for JSON and Leaflet.

No sample coordinate, candidate point, catchment boundary, area, or expected elevation is stored in the implementation.

4.4 Provided-map result

The supplied `contours_1m.kml` was processed through the deployed service on 28 August 2026. The measured output was:

Metric	Result
Contour LineStrings	1,355
Source vertices	159,113
Distinct elevation levels	32
Elevation range	267.0-298.0 m
Median contour interval	1.0 m
Analysis grid	148 x 181

Metric	Result
Cell size	18.0 m
Directly observed analysis cells	85.206%
Harmonic iterations	31, converged
Candidate coordinate	21.239822, 81.286438
Candidate interpolated elevation	271.323 m
Modelled catchment	3,921,224.77 m ² / 392.1225 ha
Catchment cells	12,103
Catchment share of study grid	46.386%

These figures demonstrate the working algorithm on the provided map; they are not a field-verified pond recommendation.

5. Location-based elevation and hydrology

The location workflow requests WGS84 elevation points through Open-Meteo. The documented source is Copernicus DEM 2021 GLO-90. Grid density is radius-aware and targets approximately 90 m cells, subject to configured limits. The public API path is capped at 23 x 23 locations to remain below its public request allowance; the response is marked degraded when this cap is coarser than the target.

If the point API fails or returns insufficient coverage, the backend uses a bounded Terrain Tiles fallback rather than fabricating elevations. The fallback downloads only the required HTTPS Terrarium PNG tiles and validates tile count, response size, media type, dimensions, channels, decoded elevation range, and coverage. Terrarium values are decoded as (red x 256 + green + blue / 256) - 32768. The result identifies the AWS Open Data/Tilezen source, reports its own resolution, and is always marked degraded.

Every batch is checked for matching length, numeric range, finite coverage, and final shape. The system does not fill a failed batch with synthetic terrain. A small percentage of isolated missing cells can be locally interpolated, while larger missing regions make elevation unavailable.

Priority-flood, D8, accumulation, catchment traversal, area calculation, and boundary extraction match the contour workflow. A radius is the half-width of the square grid, not proof that the full upstream catchment lies inside it. Roads, culverts, irrigation channels, breached embankments, structures, and DEM vertical error can materially change real drainage.

6. Rainfall analysis

The default period is 1991-2025 and the primary model is ERA5-Land through Open-Meteo. When that service is rate-limited or returns insufficient complete years, the backend requests PRECTOTCORR from NASA POWER. The fallback is identified as MERRA-2 corrected precipitation and marked degraded because its 0.5 by 0.625 degree grid is not a village rain gauge.

For both sources, daily non-negative precipitation is grouped by calendar year. Only years containing every expected date are used, so gaps cannot reduce a total as if they were zero rainfall. Response size, sentinel values, date/value alignment, numerical range, complete-year count, and coverage are validated. Monthly means state their contributing complete-year count.

Mean annual rainfall supports screening water yield. It is not a design storm and cannot size a spillway or establish flood safety.

7. Satellite surface screening and land limits

The imagery service downloads all tiles needed for the selected extent and rejects missing, malformed, blank, low-contrast, or inconsistent images. Zoom is radius-aware to bound network work and memory. Provider name, retrieval time, approximate resolution, source status, and terms link are returned.

OpenCV screens broad HSV/RGB classes for vegetation, water-like pixels, brown or sandy surface, and low-saturation surface. Morphological cleanup removes small noise. The largest qualifying region becomes a **bare-surface candidate**, and candidate placement is restricted to its intersection with the modelled catchment.

Satellite color cannot prove government ownership, legal availability, soil, rock, crops, seasonal water, utilities, protected status, or excavation suitability. Therefore the UI and API never describe this polygon as verified government land. Cadastral and field verification are mandatory.

8. Runoff and pond geometry

Annual screening water yield is:

$$V = C \times A \times P$$

where V is cubic metres per year, C is a documented runoff coefficient, A is modelled catchment area in square metres, and P is mean annual rainfall in metres. The system does not infer C from image color. If the coefficient or its basis is absent, runoff and pond geometry are unavailable rather than guessed.

The hosted course demo labels $C = 0.30$ as a demonstration scenario so the complete calculation path can be shown. It is not a site-approved coefficient and must be replaced for real work. Peak flow is separate and remains absent until an approved design rainfall intensity is configured:

$$Q = C \times i \times A / 3.6$$

for area in square kilometres and intensity in millimetres per hour.

Pond capacity uses rectangular-frustum geometry with configurable length/width ratio, side slope, water depth, freeboard, and capture efficiency. Capacity is measured to water level; excavation volume and crest dimensions include freeboard. The footprint is constrained by candidate area and no geometry is returned if even the minimum cross-section cannot fit.

The module does not design an inlet, outlet, spillway, bund, liner, sediment forebay, ramp, fencing, slope stabilization, or construction sequence.

9. Frontend and visualization

The responsive sidebar contains a Phase 2 file uploader plus place search, coordinate entry, radius selection, request start/cancel controls, and reset. The map displays satellite imagery and relevant result layers. Contour-upload results show the uploaded study boundary, computed catchment, and candidate point. Location results can show catchment, DEM contours, surface candidate, selected centre, and pond candidate.

Results include source panels, quality chips, warnings, terrain/hydrology statistics, monthly rainfall chart, land-screening ratios, runoff assumptions, and pond dimensions. Form labels, keyboard focus, high contrast, live regions, text alternatives, reduced-motion support, and responsive breakpoints improve accessibility.

10. API, errors, and security

FastAPI provides Swagger at `/docs`, ReDoc at `/redoc`, and OpenAPI JSON at `/openapi.json` when enabled. Pydantic rejects unknown fields, non-finite values, unsupported coordinates, invalid radius, and malformed response data. Error payloads use stable codes and safe messages.

Per-scope rate limits protect analysis, contour upload, search, and history. Logs contain a sanitized request ID, method, path, status, and duration, but not exact submitted coordinates or secrets. API responses use `no-store` and add anti-sniffing, frame, referrer, and permissions headers. CORS and trusted hosts are explicit in deployment. The 15 MiB upload limit is enforced before parsing and at the Nginx boundary.

Secrets remain outside version control. Production configuration rejects wildcard origins/hosts, placeholder geocoding contacts, insecure provider URLs, weak history configuration, unbased runoff values, and unconfirmed provider authorization.

11. Reliability and operations

External requests share a bounded HTTP connection pool and use timeouts, retries with capped backoff, bounded TTL caching, and source-specific validation. Liveness and readiness are separate. A database outage affects readiness only when history is enabled, and persistence failure does not erase an otherwise valid analysis response.

The backend and Nginx containers run as non-root users with health checks and restart policies. Docker Compose keeps the API and PostgreSQL off public host ports and exposes the frontend proxy on 8080. The Render Blueprint provides a free FastAPI service plus a static CDN frontend with cache and security headers.

12. Verification evidence

The final local quality matrix includes:

Gate	Result
Backend Pytest suite	60 passed
Backend statement coverage	81.99%, above 70% CI threshold
Python Ruff lint	Passed
Python dependency consistency	pip check passed
Python vulnerability audit	0 known vulnerabilities
Frontend Vitest suite	7 passed
Frontend Oxlint	Passed with 0 warnings
Vite production build	Passed
npm high-severity audit	0 vulnerabilities
Provided KML direct analysis	Passed, converged in approximately 3 seconds
Provided KML API upload	HTTP 200 with typed response
OpenAPI schema route	Passed
Docker backend build	Included in local/CI verification
Docker frontend build	Included in local/CI verification
Alembic PostgreSQL migration	Included in GitHub Actions
GitHub Actions CI on deployed commit e782496	Passed
Hosted frontend and security headers	HTTP 200; frame, MIME and referrer headers passed
Hosted API liveness and OpenAPI	HTTP 200
Hosted supplied-KML upload	HTTP 200 in 6.89 s; 392.1225 ha catchment
Hosted 2 km location analysis	35 rainfall years; 1,190.0826 ha catchment; pond branch returned

Backend tests cover contour/KMZ safety, parsing variants, KML route behavior, input limits, D8 flow, accumulation, catchment traversal, priority-flood, masking, runoff equations, frustum sizing, imagery validation, rainfall gaps, upstream failures, response quality, security defaults, rate limiting, health, and privacy defaults. Frontend tests cover explicit search, search validation, rainfall rendering, select-then-confirm analysis, and the KML upload contract.

13. Deployment and submission

The repository contains `render.yaml` for repeatable deployment in the Singapore region. The following endpoints were deployed and verified on 28 August 2026:

- Frontend: <https://sneha-village-pond-planning-2026.onrender.com>
- API: <https://sneha-village-pond-api-2026.onrender.com>
- API documentation: <https://sneha-village-pond-api-2026.onrender.com/docs>

Free Render web services can sleep after inactivity, so the first API request may require a cold-start wait. The hosted KML verification processed all 1,355 contours and 159,113 source vertices in 6.89 seconds, returning the same 392.1225 ha catchment and candidate coordinates as local and Docker tests. The final hosted location test also exercised the fallback rainfall and complete pond paths: 35 complete rainfall years, 1,324.16 mm mean annual rainfall, a 1,190.0826 ha modelled catchment, 4,727,579.5 m3 screening runoff, and a pond candidate. Its degraded quality status and field-verification warnings are intentional and visible to the user. Docker deployment and post-deploy checks are documented separately in `docs/DEPLOYMENT.md`.

14. Limitations and required field work

Before any excavation decision, obtain:

- cadastral ownership, easements, consent, and legal land availability;
- total-station, RTK/GNSS, or equivalent terrain and outlet survey;
- soil, infiltration, erodibility, lining, slope stability and geotechnical data;
- groundwater level, recharge objective and downstream-impact assessment;
- approved runoff coefficient and intensity-duration-frequency design rainfall;
- sediment, evaporation, seepage, routing and environmental-release allowances;
- utilities, buildings, roads, crops, ecology, protected areas and water quality;
- detailed civil/geotechnical design, drawings, bill of quantities and approvals.

These limits are part of the result contract and user interface rather than being hidden only in this report.

15. AI-tool usage and academic integrity

OpenAI Codex was used as an AI-assisted development tool for requirement review, code review, debugging, refactoring suggestions, automated test development, security checks, and documentation formatting. Generated suggestions were not accepted as unverified output: project files were inspected, algorithms were checked against the assignment, tests and audits were executed, failures were corrected, and the supplied KML was processed end to end. The student remains responsible for understanding and explaining every submitted component, including KML parsing, interpolation, D8 hydrology, runoff equations, pond geometry, API contracts, deployment, and limitations.

16. Conclusion

The completed project satisfies the assignment's software deliverables with a generalized Phase 2 KML/KMZ catchment API, an integrated accessible frontend, source-aware terrain/rainfall/imagery analysis, configurable runoff and pond screening, API and installation documentation, deployment infrastructure, and a repeatable automated test matrix. The main design achievement is not only producing a result, but distinguishing computed screening evidence from facts that require cadastral, survey, environmental, and engineering authority.

References

- 1 Open-Meteo Elevation API: <https://open-meteo.com/en/docs/elevation-api>
- 2 Open-Meteo Historical Weather API: <https://open-meteo.com/en/docs/historical-weather-api>
- 3 Nominatim Usage Policy: <https://operations.osmfoundation.org/policies/nominatim/>
- 4 FastAPI documentation: <https://fastapi.tiangolo.com/>
- 5 Leaflet documentation: <https://leafletjs.com/reference.html>
- 6 OpenCV documentation: <https://docs.opencv.org/>
- 7 Render Blueprint specification: <https://render.com/docs/blueprint-spec>
- 8 AWS Open Data Terrain Tiles: <https://registry.opendata.aws/terrain-tiles/>
- 9 Tilezen Terrarium format: <https://github.com/tilezen/joerd/blob/master/docs/formats.md>
- 10 NASA POWER Daily API: <https://power.larc.nasa.gov/docs/services/api/temporal/daily/>